CSU34041

Introduction to NoSQL

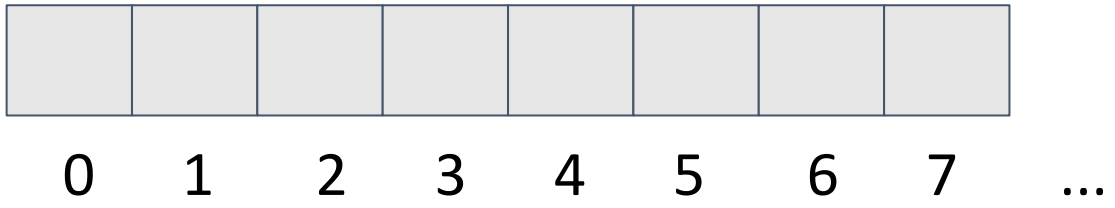
Yvette Graham
ygraham@tcd.ie

Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

What is NoSQL?

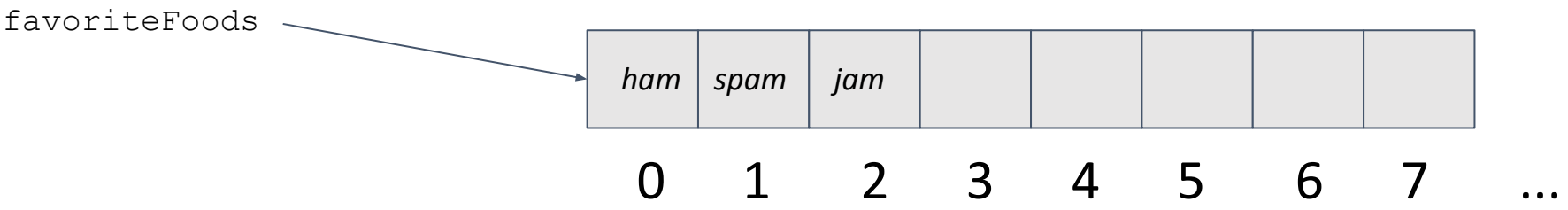
Data Structures

- A **data structure** is a particular way of **organizing data in memory** so that it can be used effectively by software/ computer programs



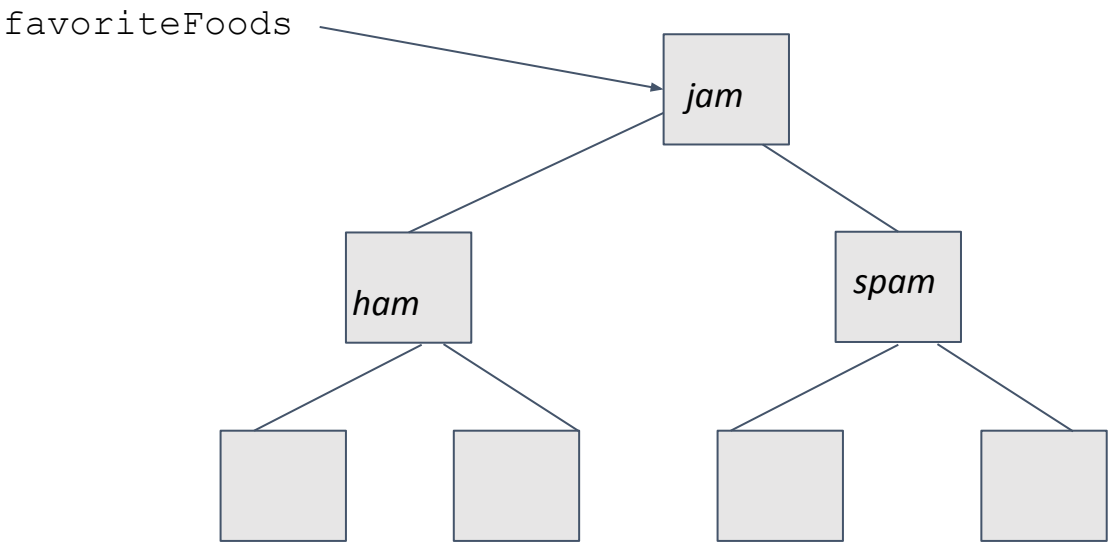
Data Structures

- A **data structure** is a particular way of **organizing data in memory** so that it can be used effectively by software/ computer programs

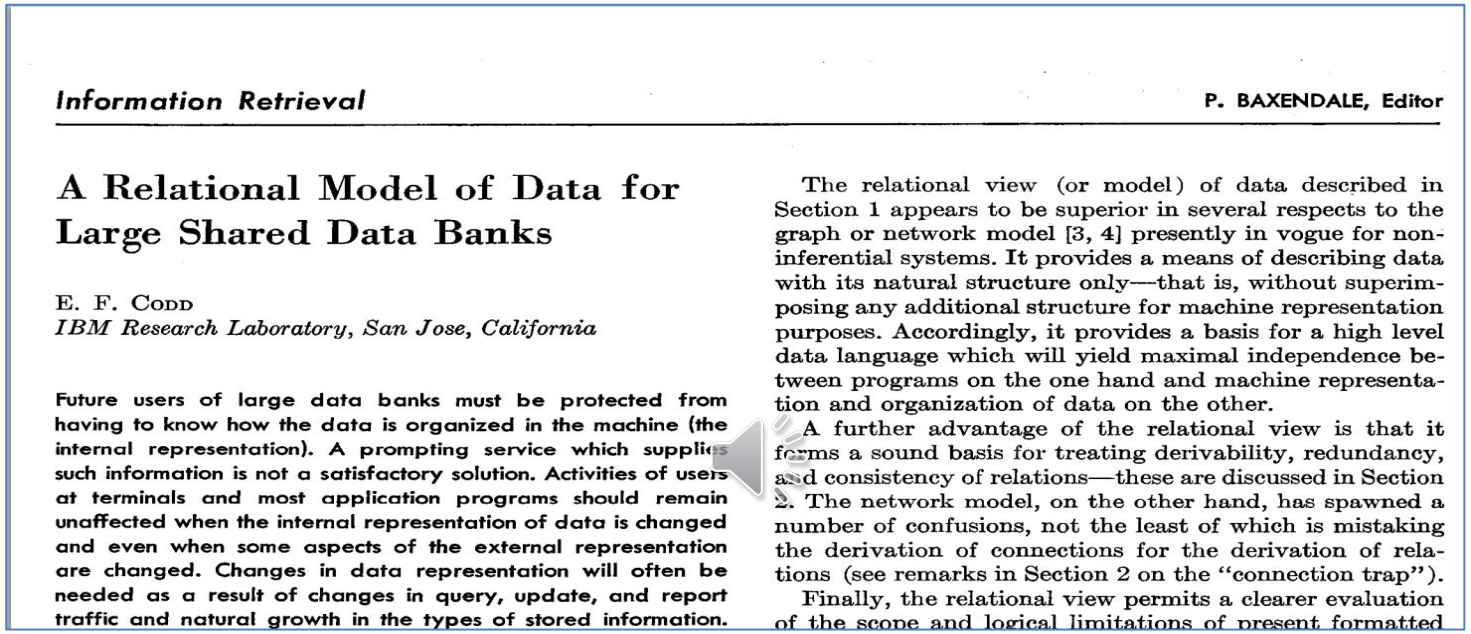


Data Structures

- A **data structure** is a particular way of **organizing data in memory** so that it can be used effectively by software/ computer programs



The relational model : 1970



Information Retrieval P. BAXENDALE, Editor

A Relational Model of Data for Large Shared Data Banks

E. F. CODD
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

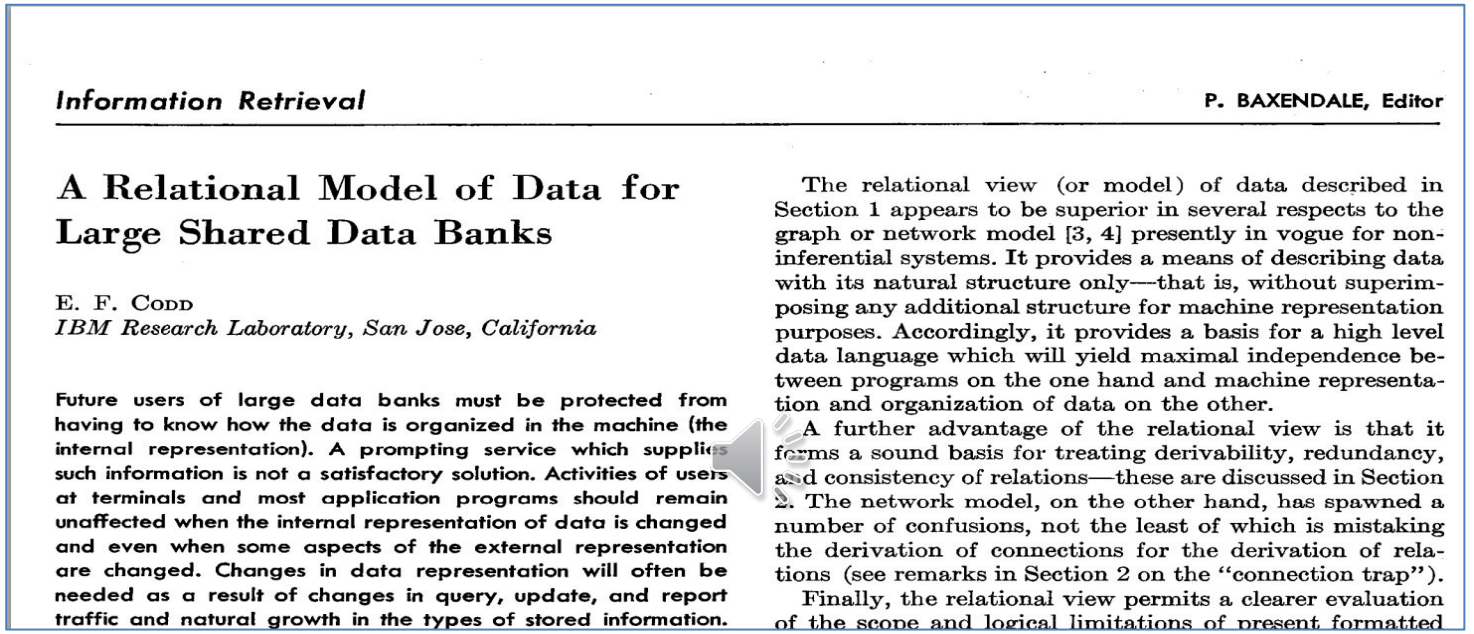
The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for non-inferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.

A further advantage of the relational view is that it forms a sound basis for treating derivability, redundancy, and consistency of relations—these are discussed in Section 2. The network model, on the other hand, has spawned a number of confusions, not the least of which is mistaking the derivation of connections for the derivation of relations (see remarks in Section 2 on the "connection trap").

Finally, the relational view permits a clearer evaluation of the scope and logical limitations of present formatted

- Since they first appearance, relational databases have been a default choice in many different context and especially in enterprise applications - Persistence + Concurrency + Integration + Almost standard model

So why do we need anything beyond Relation Databases?



Information Retrieval P. BAXENDALE, Editor

A Relational Model of Data for Large Shared Data Banks

E. F. CODD
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

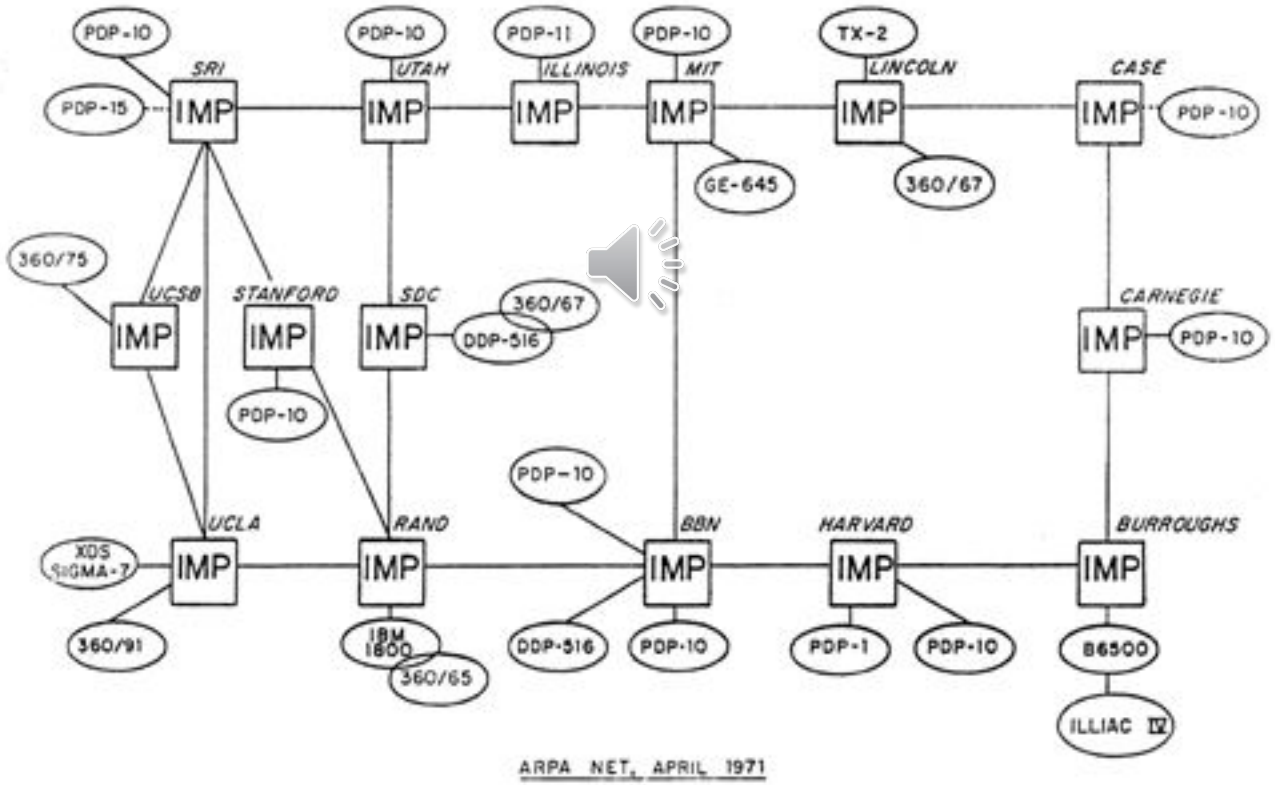
The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for non-inferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.

A further advantage of the relational view is that it forms a sound basis for treating derivability, redundancy, and consistency of relations—these are discussed in Section 2. The network model, on the other hand, has spawned a number of confusions, not the least of which is mistaking the derivation of connections for the derivation of relations (see remarks in Section 2 on the "connection trap").

Finally, the relational view permits a clearer evaluation of the scope and logical limitations of present formatted

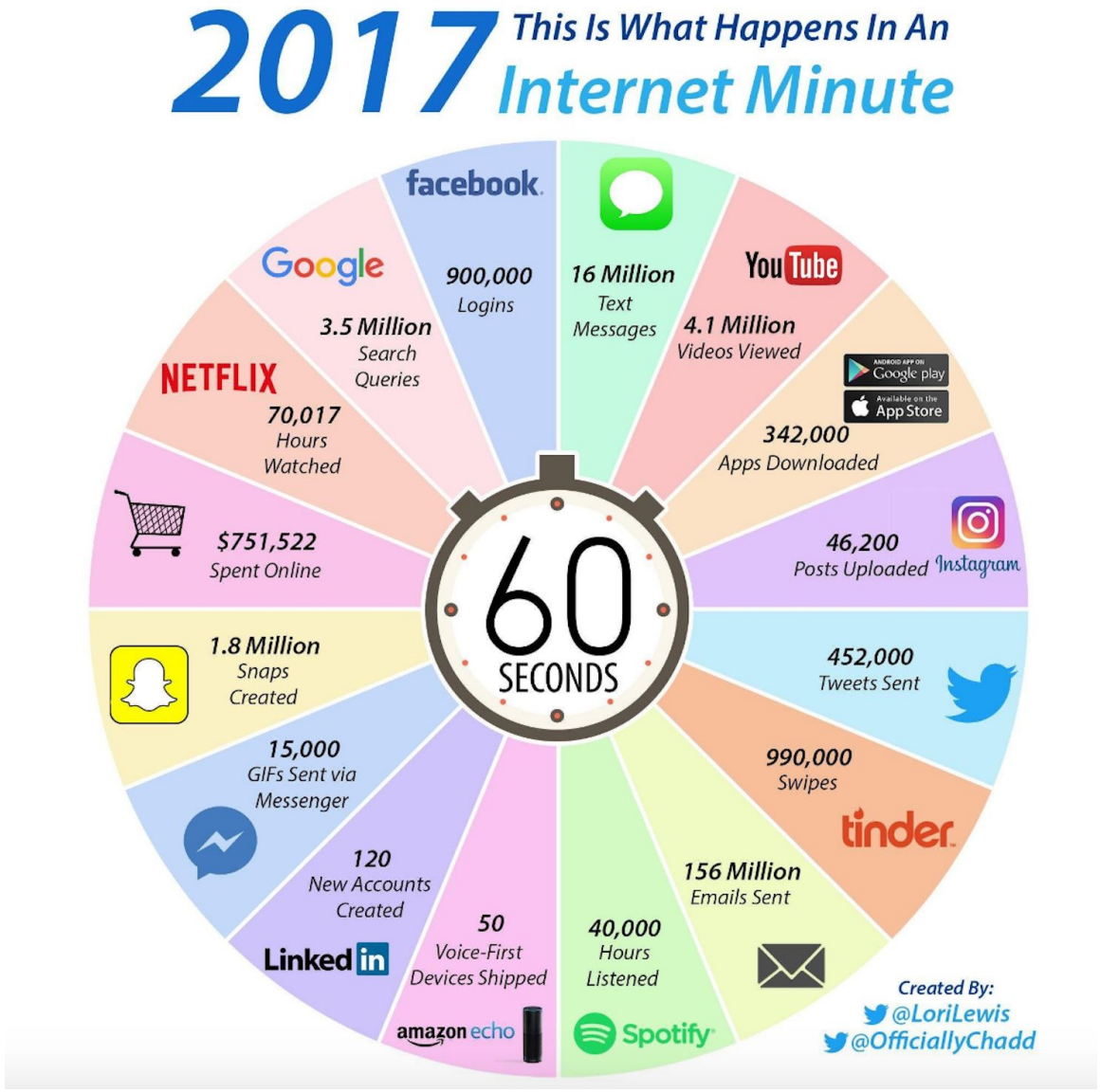
- For application developers, the biggest frustration has been what's commonly called the **impedance mismatch**: the difference between the relational model and the in-memory data structure
- No non-simplistic data structures, such as nested records or lists
- if you want to use a richer in-memory data structure, you have to translate it to a relational representation to store it on disk

The Internet - 1971



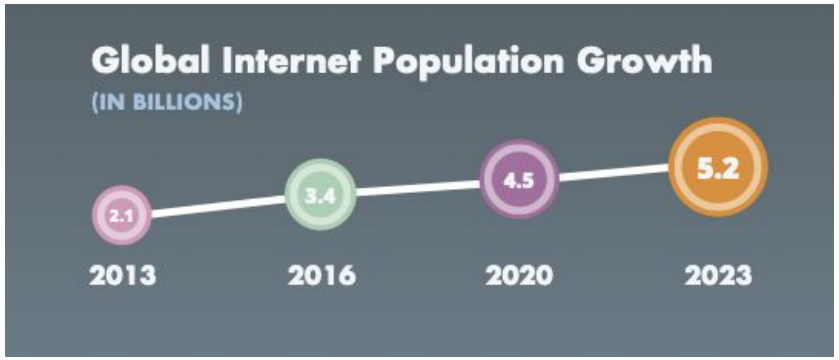
The Internet 2014

Trend 1: Data Size 2017 v 2023



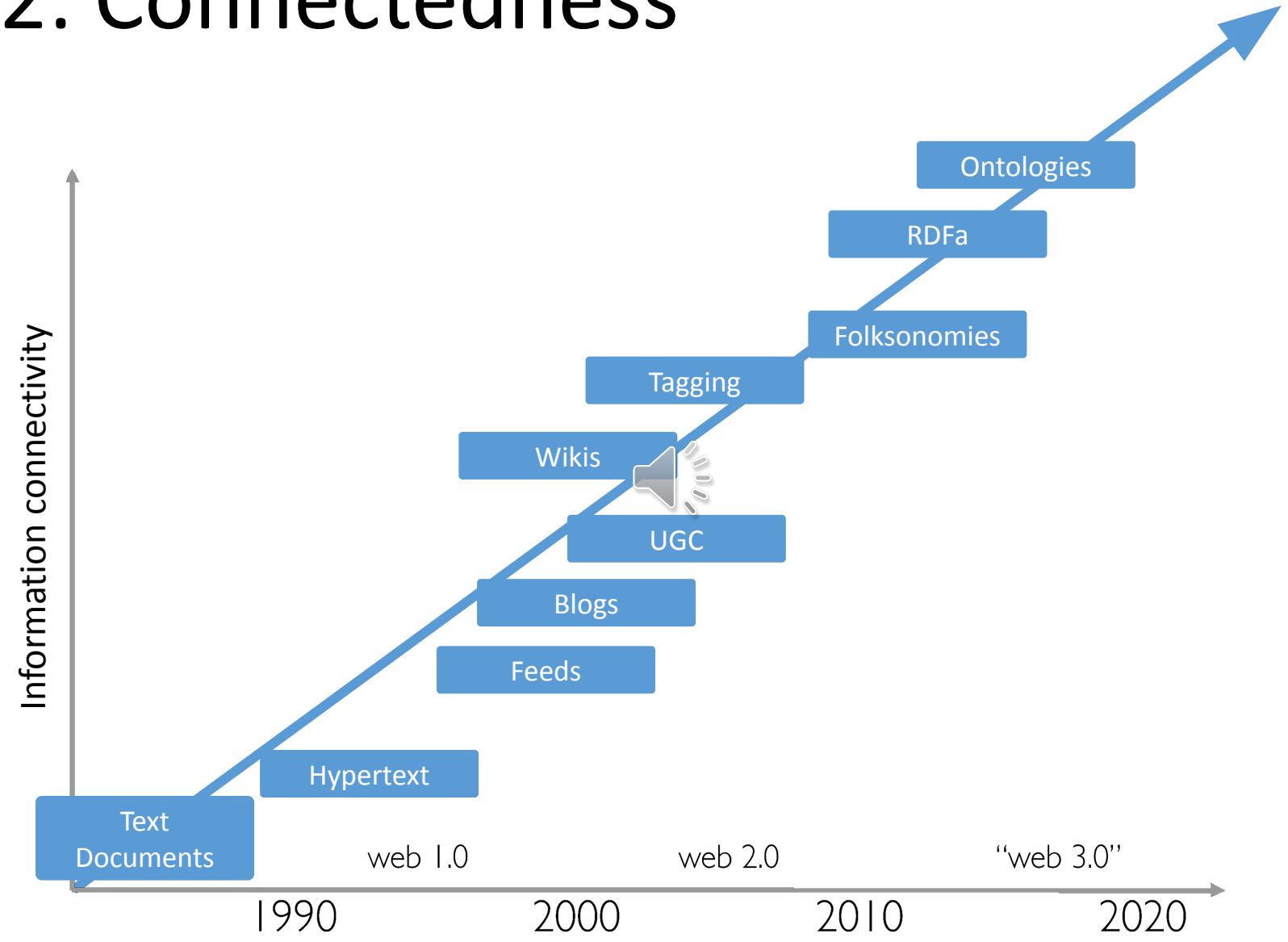
http://www.visualcapitalist.com/happens-internet-minute-2017/

Trend 1: Data Size 2025



http://www.domo.com

Trend 2: Connectedness



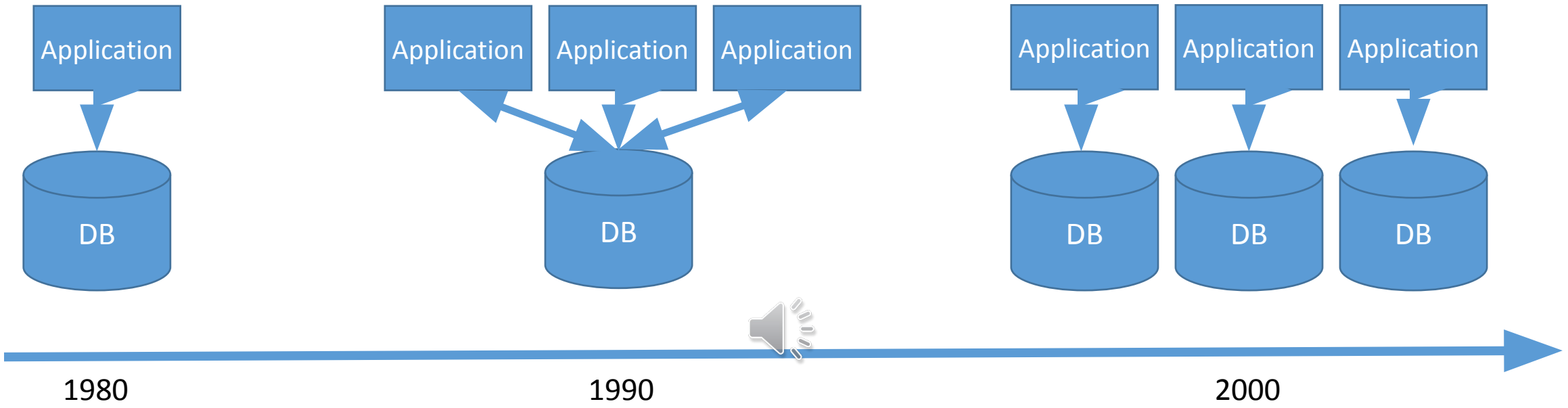
Trend 3: Semi-Structure

- Individualisation of content
  - In the salary lists of the 1970s, all elements had exactly one job
  - In the salary lists of the 2000s, we need 5 job columns! Or 8? Or 15?

- All encompassing “entire world views”
  - Store more data about each entity

- Trend accelerated by the *decentralization* of *content generation* that is the hallmark of the age of participation (“web 2.0”)

Trend 4: Architecture



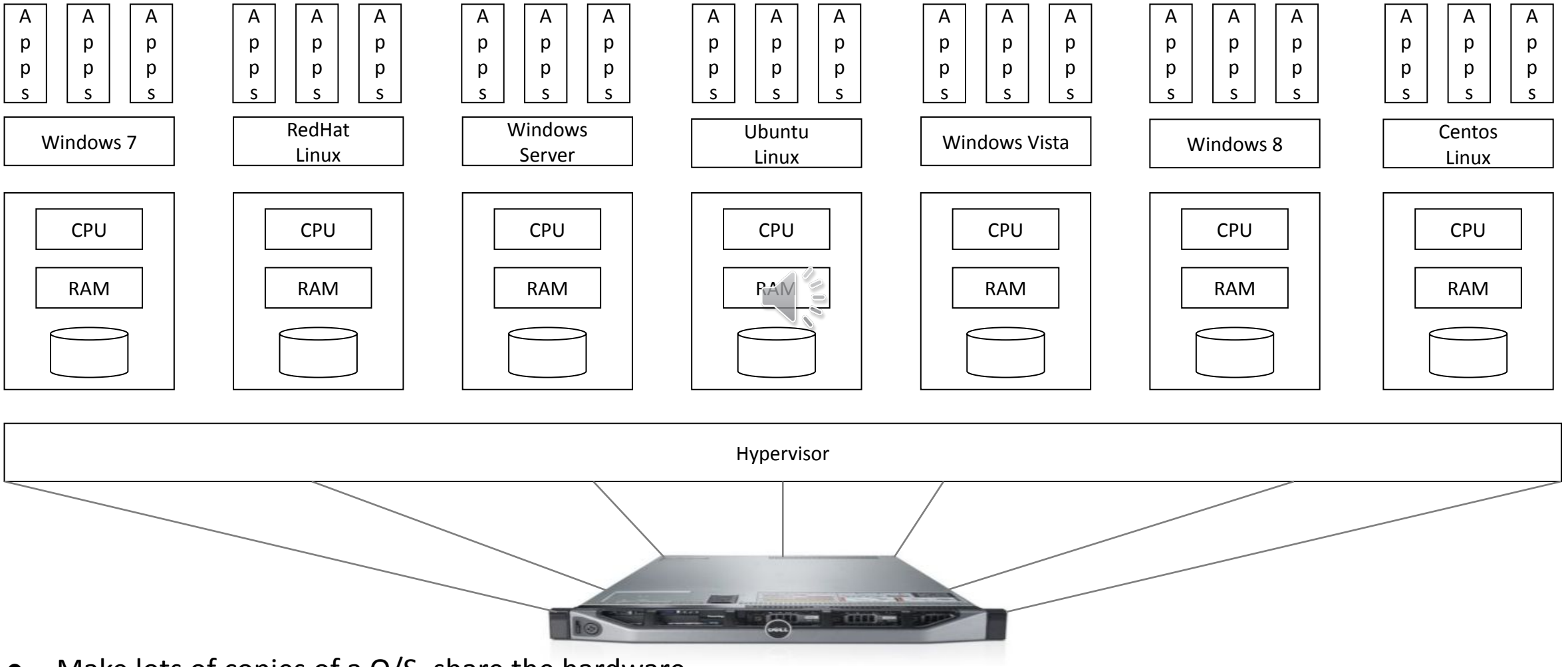
Mainframe application

Database as Integration hub

Decoupled services

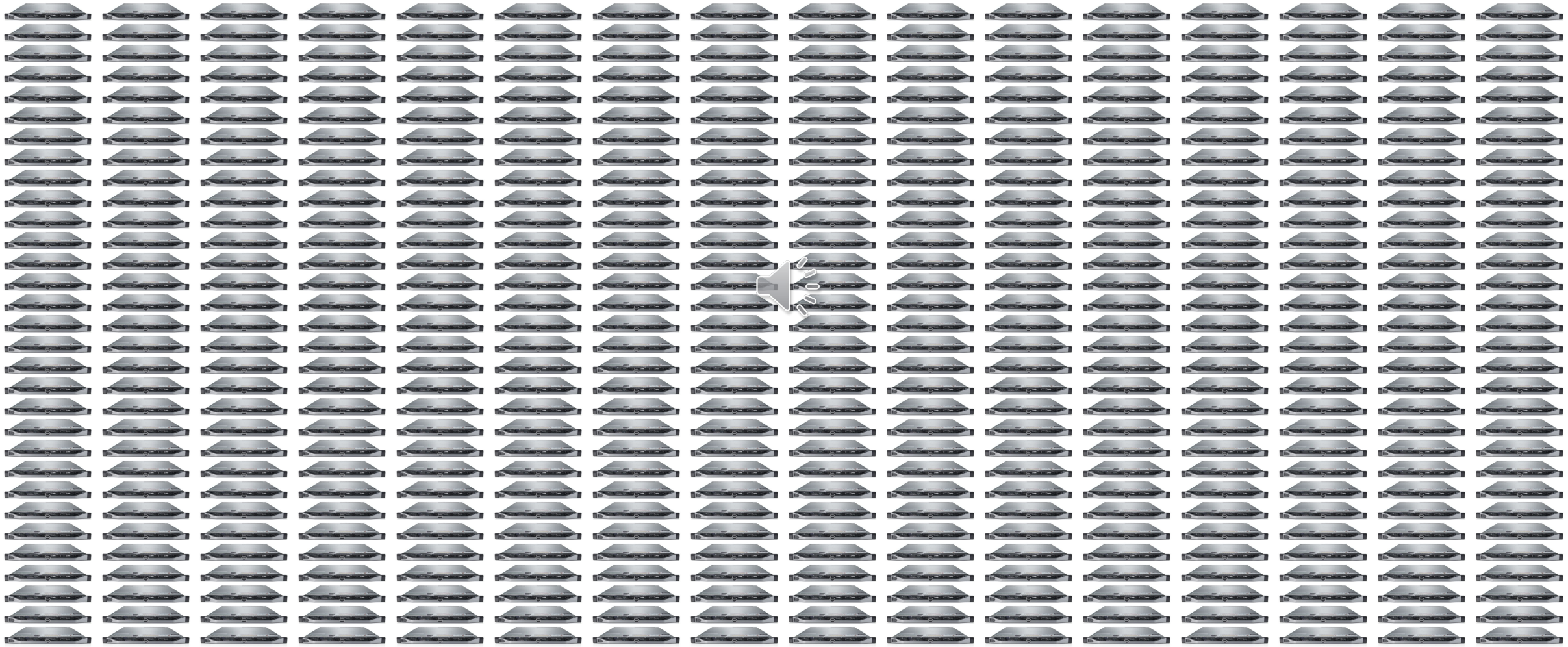
Multicore
Parallelization/distributed
Cloud
Schema less

Virtualization

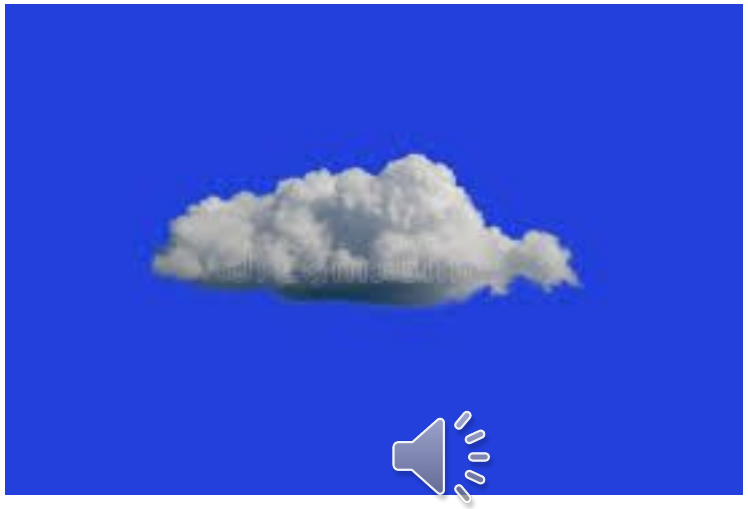


- Make lots of copies of a O/S. share the hardware
- Each virtual machine runs its own operating system and functions separately from the other VMs, even when they are all running on the same host

Now Imagine Lots



Cloud Computing



- Takes virtualization to the extreme
- Companies like Google, Amazon and Microsoft have over 500,000 physical servers
- Anyone with a credit card can start hundreds of servers in a matter of minutes
- Especially used for data storage and computing power, without direct active management by the user

Why NoSQL

- **Explosion** of (unstructured) data
- Big data is data that **exceeds the processing capacity** of conventional database systems
- The data is too big, moves too fast, or **doesn't fit** the structures of your database architectures
- To gain value from this data, you need **alternative way to process it**

Why NoSQL

- **Big data**, became viable as cost-effective approaches have emerged to tame the volume, velocity and variability of massive data
- Within this data lie valuable **patterns** and **information**, previously hidden because of the amount of work required to extract them
- Here we are storing **huge amount of data** - need a processing system to handle and analyse the data in efficient manner
- Data is growing
- We need to handle the huge **Volume** and **Variety** of data with **Velocity - 3 Vs**

Unlock Your Big Data

Traditional Data Approaches

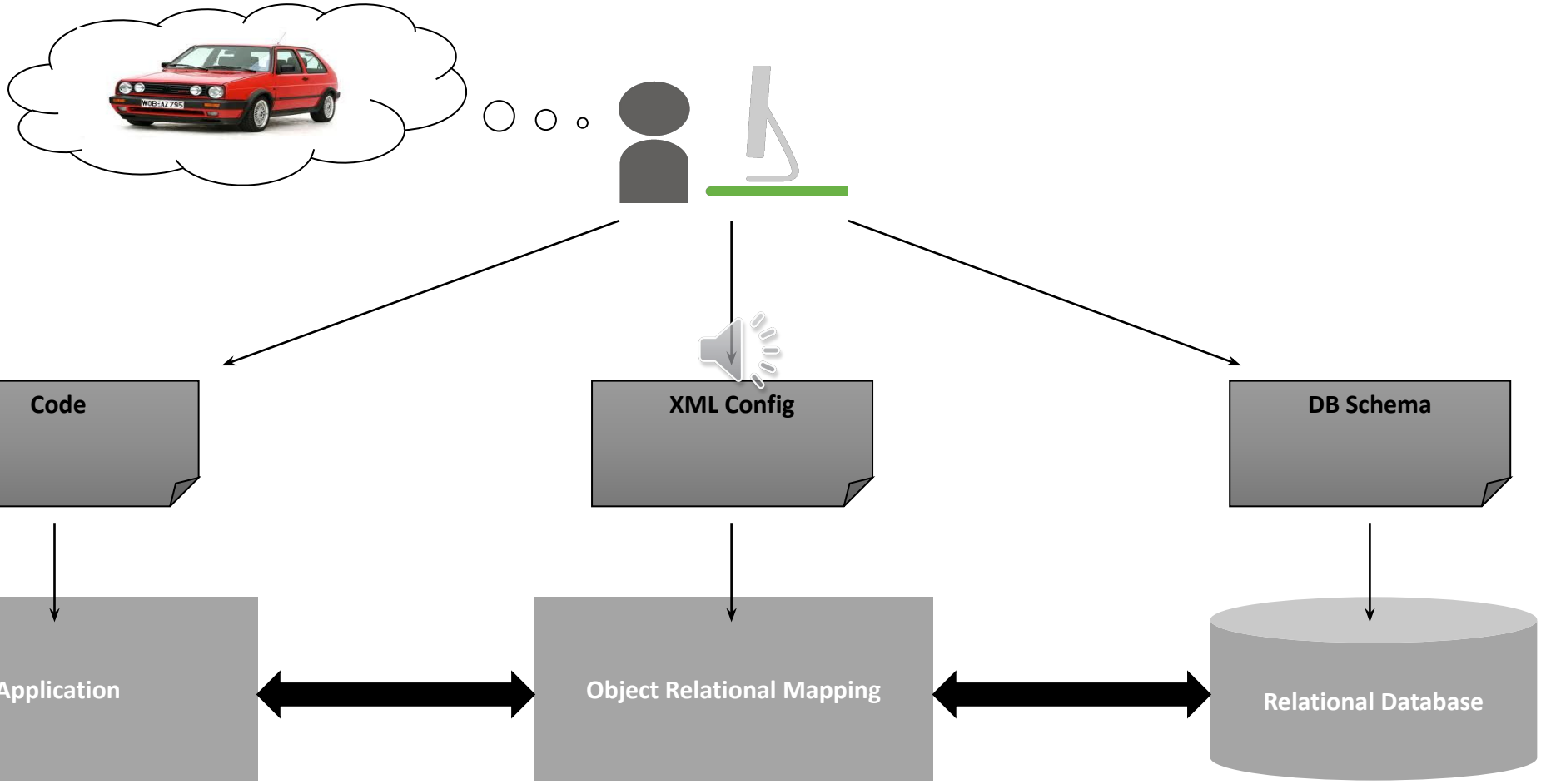
- **Filter – Store – Distribute**
  - Encyclopedias
  - Newspapers
  - Libraries
  - Banking
- Why?
  - Storage caps
  - Bandwidth caps

Storing Everything Is A Challenge

SQL Databases

- Depend on a pre-filter
- Assume single disk farm
- Hard to partition
- Based on 1970’s storage assumptions

This makes software development difficult - **impedance mismatch**: the difference between the relational model and the in-memory data structures



Limitations of Relational Databases

- Impedance Mismatch - complex objects are not suited to being represented in a relational way
- Application and integration - the database works as an integration database. However, in this scenario the structure of the database tend to be more complex

- Scale up vs. scale out (parallel)

What did possibility of scaling out result in?

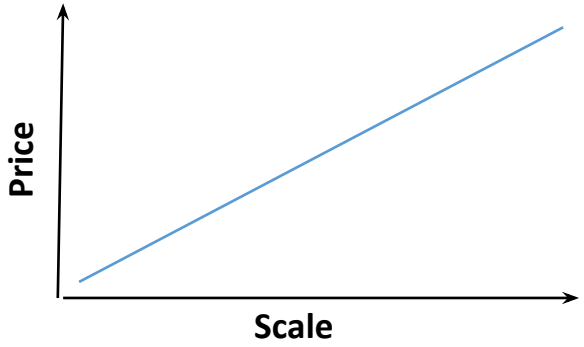
- No CAPEX - Capital expenditure - funds used by a company to acquire or upgrade physical assets such as property, industrial building or equipment
- No Data Centre
- Availability of Scale
- Utility Pricing (pay per use)

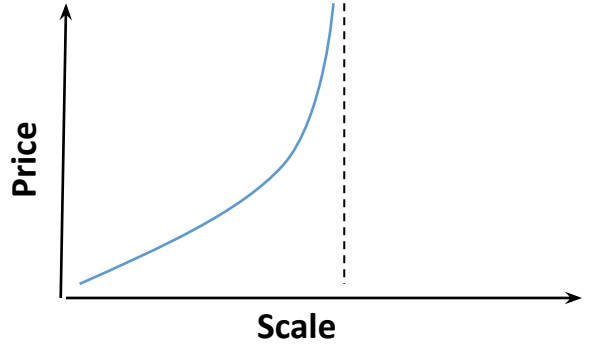
Filter-Store-Distribute

Store-Filter-Distribute

NoSQL Scales Better

Vs.





When?

What?

Not Only SQL

NoSQL – No Definition

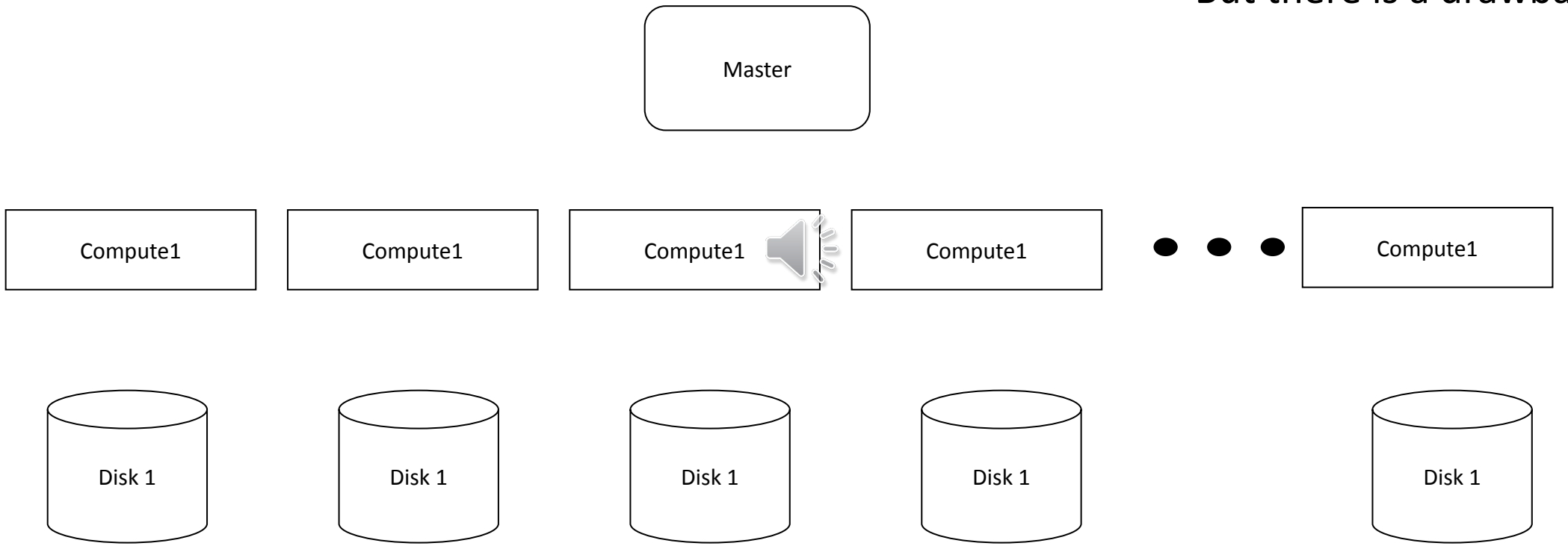
- Non Relational
- No SQL as query language - They don’t use SQL as query language, although some of them may have some form of query language that resembles SQL
- Schema less - structure can change
- Usually –not always – Open-Source project
- They are distributed: they are usually driven by the requirements of running on clusters (with the exception of graph DBs)
- RDMS use ACID transactions, NoSQL don’t

What is NoSQL?

- Goal of a Database
  - Data Durability
  - Consistent performance
  - Graceful degradation under load
- Big Data Workloads require distributed computing
  - ~~Transactions~~
  - ~~Joins~~

Distributed Computing

- Much more power
- Cheaper
- More resilience
- But there is a drawback -



In distributed system we have a network of autonomous computers that communicate with each other in order to achieve a goal- The computers in a distributed system are independent and do not physically share memory or processor

Distributed Models

Data Distribution

- **Replication** takes the same data and copies it over multiple nodes

- **Sharding** puts different data on different nodes

- Replication and Sharding can be used in combination or alone

Sharding

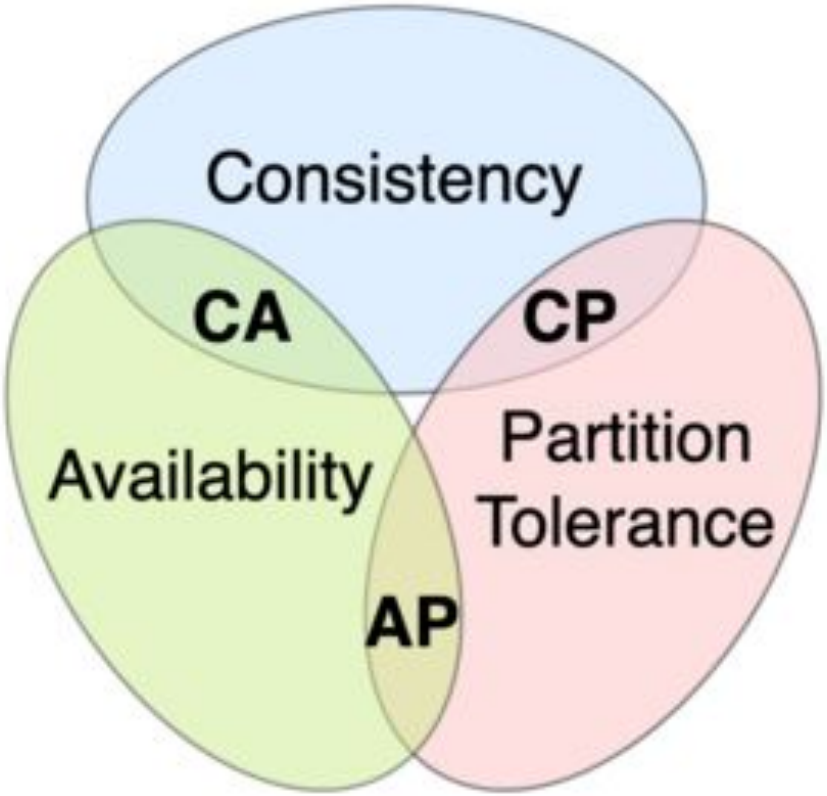
- Different data on different nodes (horizontal scalability)
- Each server acts as a single source for the subset of data it is responsible for
- Ideal setting: one user talks with one server
  - Data accessed together are stored together
  - Example: access based on physical location, place data to the nearest server
- Many NoSQL databases offer auto-sharding
- Scales read and write on the different nodes of the same cluster
- No resilience if used alone: node failure => data unavailability

Replication (combined also possible)

- The same data is replicated and copied over multiple nodes
- Master-slave: one node is the primary responsible for processing the update to data while the other are secondaries used for read operations
  - Scaling by adding slaves
  - Processing incoming data limited by master
  - Read resilience
  - Inconsistency problem (read)
- Peer-to-peer: all replicas have equal weight and can accept writing
  - Scaling by adding nodes
  - Node failure without losing write capability
  - Inconsistency problem (write)

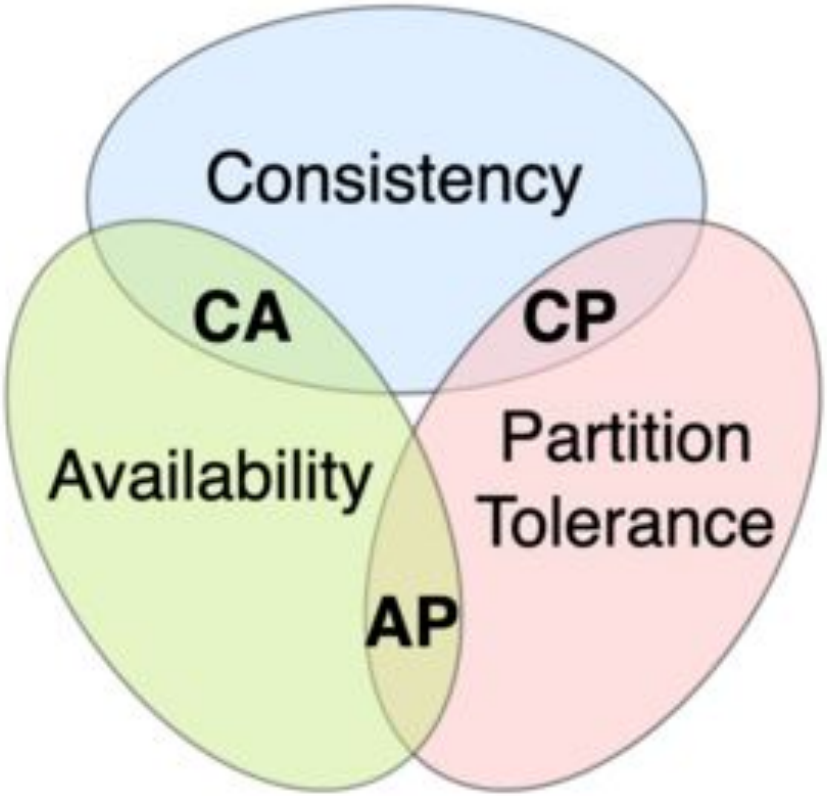
CAP Theorem

CAP Theorem - only 2 of can be guaranteed



- **Consistency**
  - All nodes see the same data at the same time
- **Availability**
  - A guarantee that every request receives a response about whether it succeeded or failed
- **Partition tolerance**
  - The system continues to operate despite arbitrary partitioning due to network failures

CAP Theorem - only 2 of can be guaranteed



When a network partition failure happens, it must be decided whether to do one of the following:

- cancel the operation and subsequently decrease the **availability** but ensure **consistency**
- proceed with the operation and thus provide **availability** but risk **inconsistency**.

Eventual Consistency

- **Eventual consistency** is a consistency model used in distributed computing to achieve high availability that informally guarantees that, if no new updates are made to a given data item, eventually all accesses to that item will return the last updated value
- Reconciliation is a problem - choosing an appropriate final state when concurrent updates have occurred, called **reconciliation**

SQL vs. NoSQL

ACID

- **A**tomic: Everything in a transaction succeeds or the entire transaction is rolled back
- **C**onsistent: A transaction cannot leave the database in an inconsistent state
- **I**solated: Transactions cannot interfere with each other
- **D**urable: Completed transactions persist, even when servers restart etc.

BASE

- **B**asic **A**vailability: An application works basically all the time
- **S**oft-state: It does not have to be consistent all the time
- **E**ventual consistency: It will be in some known-state state eventually

Each node is always available to serve requests. As a trade-off, data modifications are propagated in the background to other nodes. The system may be inconsistent, but the data is still largely accurate.

Data Model

Data Model

- A data model is a representation of how we perceive and manipulate our data
- The data model describes how we interact with the data
  - Represents the data elements under analysis
  - How these elements interact with each others

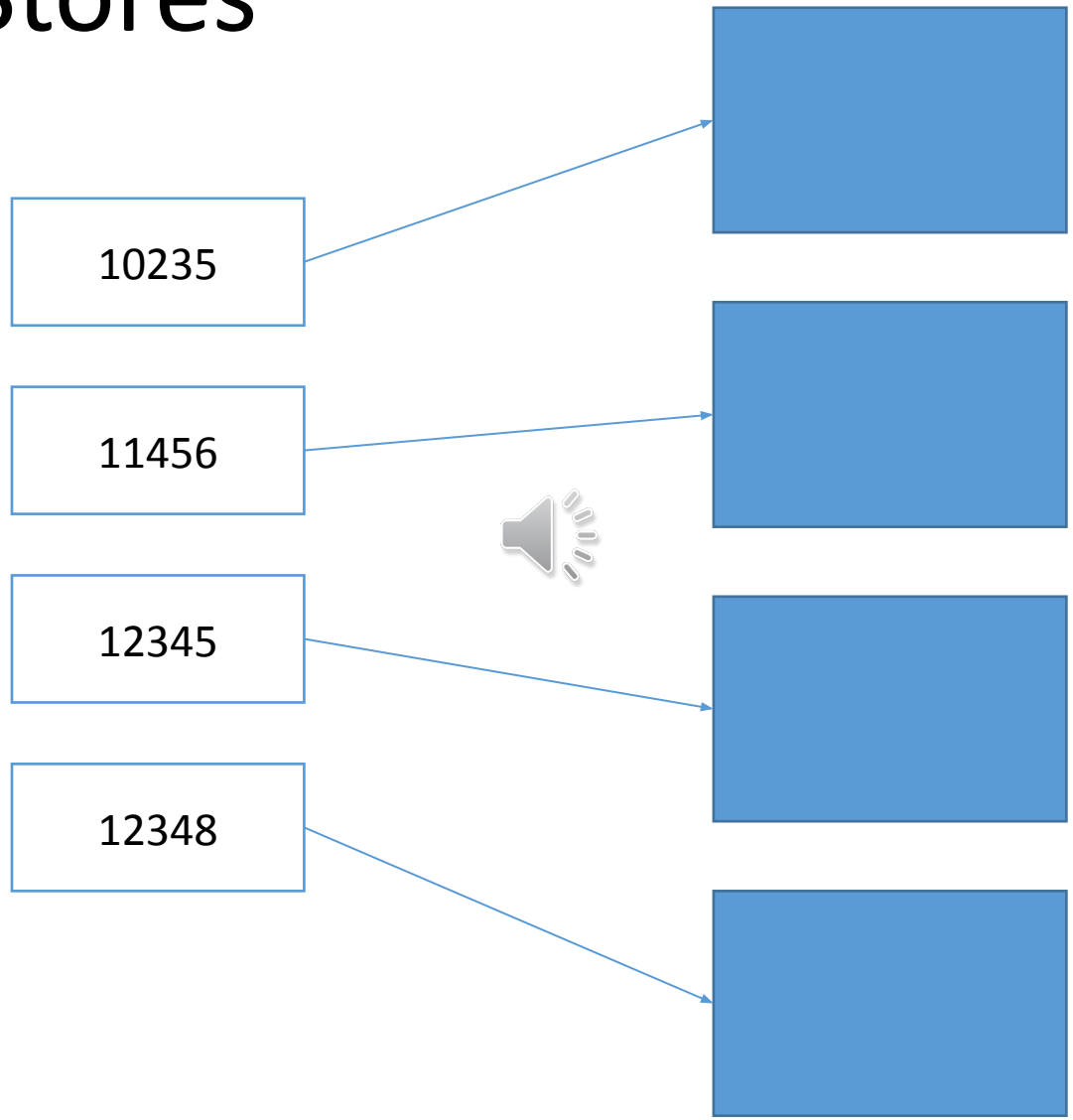
- The storage model describes how the database stores and manipulates the data internally

Four Common Types of NoSQL

- Key Value Stores
- Document Stores
- Column Stores
- Graph Stores

- Note
  - Lots of Hybrids

Key-Value Stores



Key-Value Stores

- Maps keys to values
- Values treated as a blob
  - They can be complex compound objects (list, maps, or other structures)
- Single Index
- Consistency applicable for operations on a single key
- Very fast and scalable
- Inefficient to do aggregate queries, “all the carts worth $100 or more” or to represent relationships between data
- Great for shopping carts, user profiles and preferences, storing session information

Document Databases

```
{“id”: 10203,
 ”name”:”Sara”,
 “surname”: “Parker”,
 “items”: [
 {“product_id”: 23, “quantity”: 2},
 {“product_id”: 45, ”quantity”: 1}]}
```

```
{“id”: 10456,
 ”fullName”:”John Smith”,
 “items”: [
 {“product_id”: 24, “quantity”: 4},
 {“product_id”: 45, ”quantity”: 1},
 {“product_id”: 67, ”quantity”: 34}],
 ”discount-code”: “Yes”}
```

Document Databases

- A document is like an hash, with one id and many values
- Store Javascript Documents
  - JSON = JavaScript Object Notation
  - An associative array
  - Key value pairs
  - Values can be documents or arrays
  - Arrays can contain documents

- Data is implicitly denormalised - closer to a single table than lots of tables with relations connecting them
- Document databases allow indexing of documents on the basis of not only its primary identifier but also its properties

Documents are easier

Relational

Person:

Pers_ID	Surname	First_Name	City
0	Miller	Paul	London
1	Ortega	Alvaro	Valencia
2	Huber	Urs	Zurich
3	Blanc	Gaston	Paris
4	Bertolini	Fabrizio	Rom

no relation

Car:

Car_ID	Model	Year	Value	Pers_ID
101	Bentley	1973	100000	0
102	Rolls Royce	1965	330000	0
103	Peugeot	1993	500	3
104	Ferrari	2005	150000	4
105	Renault	1998	2000	3
106	Renault	2001	7000	3
107	Smart	1999	2000	2

Document DB

```
{
  first_name: 'Paul',
  surname: 'Miller'
  city: 'London',
  location: [45.123,47.232],
  cars: [
    { model: 'Bentley',
      year: 1973,
      value: 100000, … },
    { model: 'Rolls Royce',
      year: 1965,
      value: 330000, … }
  ]
}
```

Document DB Features

Rich Queries	• Find Paul's cars • Find everybody who owns a car built between 1970 and 1980
Geospatial	• Find all of the car owners in London
Text Search	• Find all the cars described as having leather seats
Aggregation	• What's the average value of Paul's car collection
Map Reduce	• For each make and model of car, how many exist?

MongoDB

```
{
  first_name: 'Paul',
  surname: 'Miller',
  city: 'London',
  location: [45.123,47.232],
  cars: [
    { model: 'Bentley',
      year: 1973,
      value: 100000, … },
    { model: 'Rolls Royce',
      year: 1965,
      value: 330000, … }
  ]
}
```

Column Stores

- Store data as columns rather than rows
  - Columns organized in column family
  - Each column belongs to a single column family
  - Column acts as a unit for access
  - Particular column family will be accessed together

- Efficient to do column ordered operations
- Not so great at row based queries
- Adding columns is quite inexpensive and is done on a row-by-row basis
- Each row can have a different set of columns, or none at all, allowing tables to remain sparse without incurring a storage cost for null values

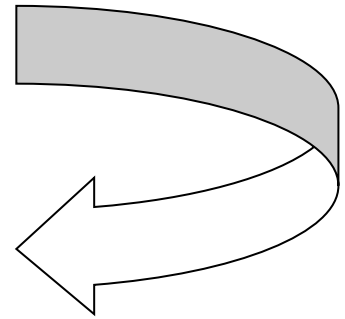
Relational/Row Order Databases

ID	Name	Salary	Start Date
1	Joe D	$24000	1/Jun/1970
2	Peter J	$28000	1/Feb/1972
3	Joe D	$23000	1/Jan/1973

Column Databases

- Materialise storage as columns, primary key is the data, which is mapped from rowids

ID	Name	Salary	Start Date
1	Joe D	$24000	1/Jun/1970
2	Peter J	$28000	1/Feb/1972
3	Joe D	$23000	1/Jan/1973



Joe D:01;03

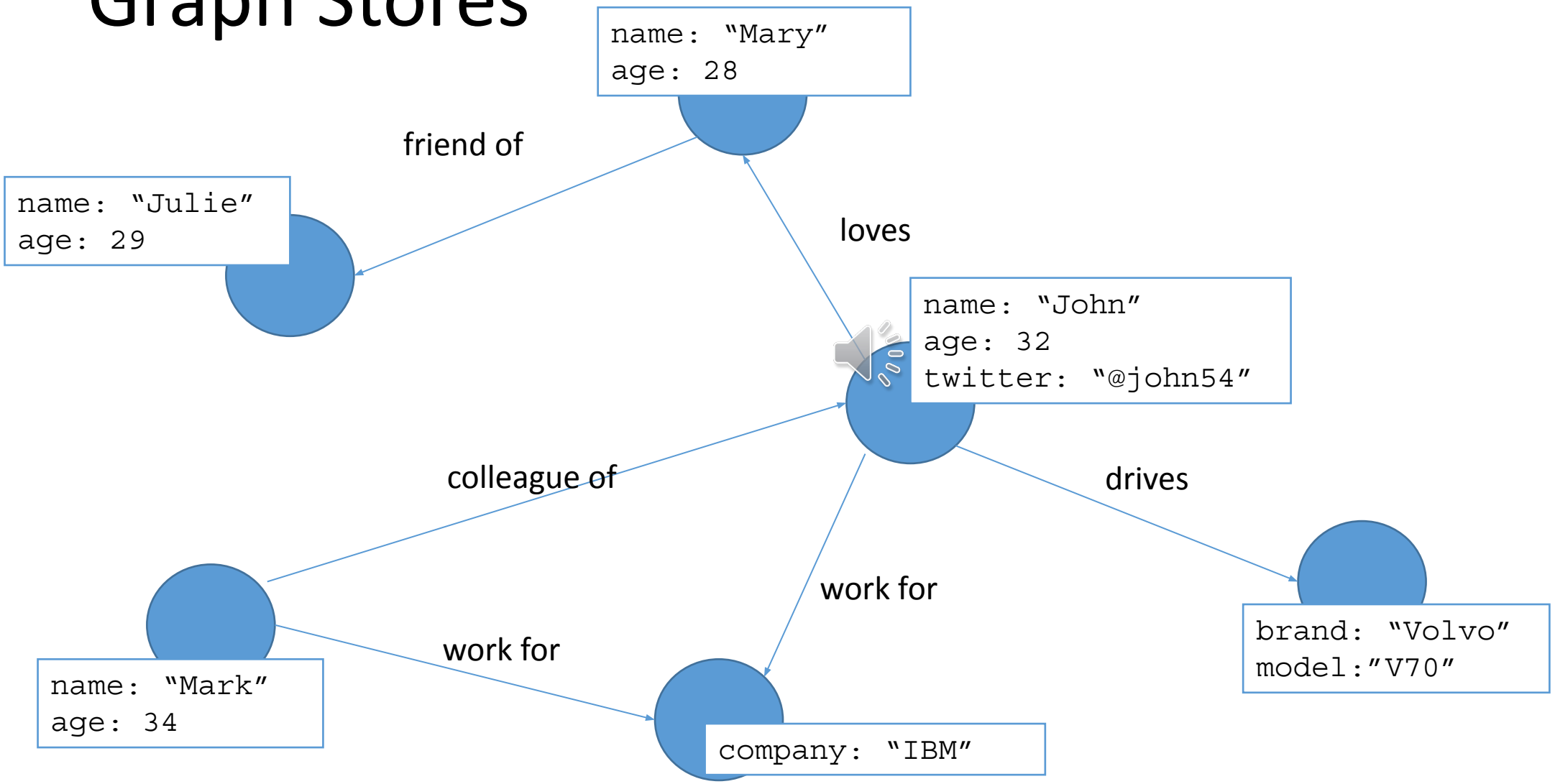
Joe D:01	Peter J:02	Joe D:03
24000:01	28000:02	23000:03
1/Jun/1970:01	1/Feb/1972:02	1/Jan/1973:03

Pros and Cons

- Relational: Good For
  - Queries that return small subsets of rows
  - Queries that use a large subset of row data
  - e.g. *find all employee data for employees with salary > 12000*

- Column: Good For
  - Queries that require just a column of data
  - Queries that require a small subset of row data
  - E.g. *Give me the total salary outlay for all staff*

Graph Stores



Graph Stores

- Data model composed by nodes connected by edges
  - Nodes represent entities
  - Edges represent the relationships between entities
  - Nodes and edges can have properties

- Querying a graph database means *traversing* the graph by following the relationships

- Pros:
  - Representing objects of the real world that are highly interconnected
  - Traversing the relationships in these data models is cheap

Graph Stores vs Relational Databases

- Relational databases are not ideally suited to representing relationships

- Relationship implemented through foreign keys
  - Expensive joins required to navigate relationships
  - Poor performance for highly connected data models

NoSQL vs. Relational Databases

Pros

- Flexible schema
- Simple API
- Scalable
- Distributed and Replicated storage
- Cheap

Cons

- Not ACID compliant
- No standards
- Eventual consistency
- Some products are at early stage: poor documentation/support

CSU34041

Introduction to NoSQL

Yvette Graham
ygraham@tcd.ie

Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin